

Mastering Data Visualization with Python

Documentation Guide on Matplotlib and Seaborn

ShadowFox Data Science Internship

Prepared By: Ananya Mandal

Course: B.Tech – Artificial Intelligence & Data Science

Internship: ShadowFox Data Science Internship

Libraries Covered: Matplotlib and Seaborn

Submission Date: July 2026

Table of Contents:

1. Introduction
2. Why Data Visualization Matters
3. Overview of Matplotlib
4. Installing and Importing Matplotlib
5. Matplotlib Graphs
 - Line Plot
 - Scatter Plot
 - Bar Chart
 - Histogram
 - Pie Chart
 - Box Plot
 - Area Chart
6. Overview of Seaborn
7. Installing and Importing Seaborn
8. Seaborn Graphs
 - Scatter Plot
 - Line Plot
 - Bar Plot
 - Count Plot
 - Histogram
 - Heatmap
 - Box Plot
 - Violin Plot
 - Pair Plot
9. Comparison between Matplotlib and Seaborn
10. Conclusion
11. References

1. Introduction

Data is generated every day through activities such as online shopping, social media, healthcare, and education. However, raw data in tables can be difficult to understand. Data visualization helps by presenting information through charts and graphs, making it easier to analyze and interpret.

In data science, visualization is used to identify patterns, trends, and relationships within data while also making insights easier to communicate. Python offers several visualization libraries, with Matplotlib and Seaborn being two of the most popular. This documentation explores these libraries, their features, common graph types, and their practical applications to help beginners get started with data visualization.

2. Why Data Visualization Matters

Data visualization makes it easier to understand large amounts of data by presenting it in the form of charts and graphs instead of rows and columns. It helps identify patterns, trends, and unusual values that might be difficult to notice from raw data alone.

Visualizations are widely used in business, healthcare, finance, and research to communicate information clearly and support better decision-making. For data scientists, visualization is an essential step in exploring data and presenting meaningful insights.

3. Overview of Matplotlib

Matplotlib is one of the most popular Python libraries for data visualization. Developed by John Hunter in 2002, it is widely used in data science, research, and business analytics to create clear and informative graphs.

Built on top of NumPy, Matplotlib offers a high level of customization, allowing users to control colors, labels, fonts, and layouts. It supports various chart types such as line plots, bar charts, scatter plots, histograms, pie charts, and box plots. Although it may require a little more code than some other libraries, its flexibility makes it an excellent choice for creating professional-quality visualizations.

3. Overview of Matplotlib

Matplotlib is one of the most popular Python libraries for data visualization. Developed by John Hunter in 2002, it is widely used in data science, research, and business analytics to create a variety of charts and graphs.

Built on top of NumPy, Matplotlib offers a high level of customization, allowing users to modify colors, labels, legends, and layouts. It supports graphs such as line plots, bar charts, scatter plots, histograms, pie charts, and box plots. Its flexibility and reliability have also made it the foundation for other libraries like Seaborn.

Key Features of Matplotlib

- Supports different types of charts and graphs.
- Highly customizable with colors, labels, and styles.
- Works well with NumPy and Pandas.
- Creates high-quality visualizations.
- Suitable for both simple and advanced plotting.

Advantages of Matplotlib

Some major advantages of Matplotlib are:

- Easy to integrate with Python libraries.
- Excellent for creating customized charts.
- Large community support and documentation.
- Ideal for academic research and data analysis.
- Suitable for beginners as well as advanced users.

4. Installing and Importing Matplotlib

Before creating visualizations, Matplotlib needs to be installed and imported into the Python environment. Google Colab already includes Matplotlib by default, but the installation command is shown below for reference.

```
In [37]: !pip install matplotlib
```

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.23 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
```

Importing Required Libraries

The following libraries are used throughout this documentation:

- **Matplotlib** – for creating visualizations.
- **Pandas** – for handling datasets.
- **NumPy** – for numerical operations.

```
In [38]: import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

Creating a Sample Dataset

To demonstrate different types of visualizations, a simple student performance dataset is created. This dataset includes students' marks, study hours, attendance, and participation in extracurricular activities.

```
In [39]: import pandas as pd

data = {
    "Student": ["A", "B", "C", "D", "E", "F"],
    "Marks": [85, 92, 78, 88, 95, 80],
    "Study_Hours": [2, 4, 3, 5, 6, 3],
    "Attendance": [90, 95, 85, 92, 98, 88],
    "Activity": ["Yes", "Yes", "No", "Yes", "No", "Yes"]
}

df = pd.DataFrame(data)

df
```

```
Out[39]:
```

	Student	Marks	Study_Hours	Attendance	Activity
0	A	85	2	90	Yes
1	B	92	4	95	Yes
2	C	78	3	85	No
3	D	88	5	92	Yes
4	E	95	6	98	No
5	F	80	3	88	Yes

5. Matplotlib Graphs

5.1 Line Plot

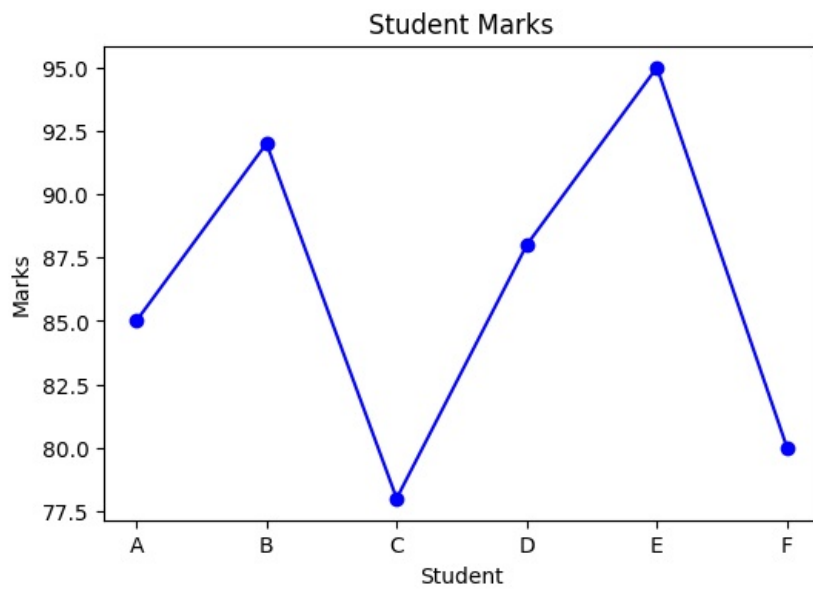
Description

A line plot displays data points connected by straight lines. It is commonly used to show changes or trends over a period or across ordered categories.

Use Case

Line plots are useful for tracking values such as monthly sales, stock prices, temperatures, or student performance over time.

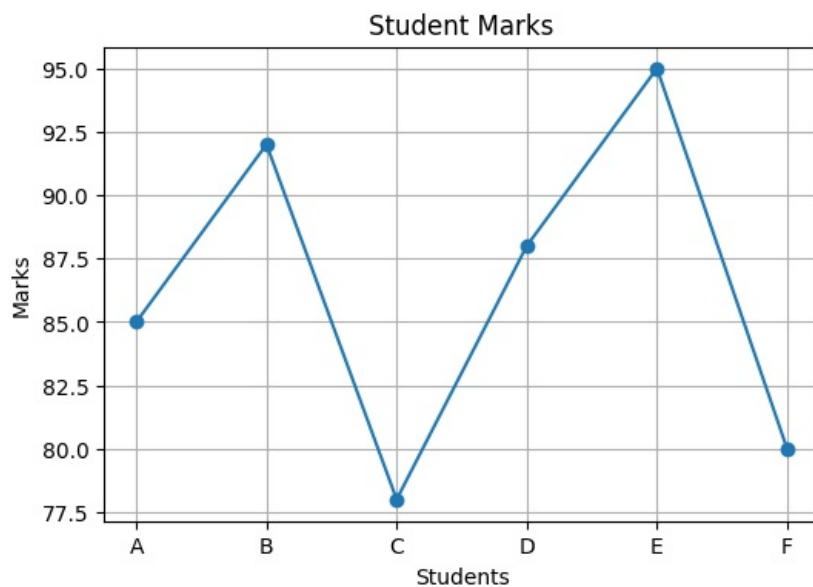
```
In [40]: plt.figure(figsize=(6,4))
plt.plot(df["Student"], df["Marks"],marker="o",color="blue")
plt.title("Student Marks")
plt.xlabel("Student")
plt.ylabel("Marks")
plt.show()
```



Observation

The line plot shows the marks scored by each student. Student E achieved the highest marks, while Student C scored the lowest. The graph makes it easy to compare the performance of all students.

```
In [41]: plt.figure(figsize=(6,4))
plt.plot(df["Student"], df["Marks"], marker="o")
plt.title("Student Marks")
plt.xlabel("Students")
plt.ylabel("Marks")
plt.grid(True)
plt.savefig("line_plot.png")
plt.show()
```



5.2 Scatter Plot

Description

A scatter plot displays individual data points on a graph to show the relationship between two numerical variables. It helps identify patterns, trends, clusters, or outliers within the data.

Use Case

Scatter plots are commonly used to analyze relationships between variables, such as study hours and marks, height and

weight, or advertising expenditure and sales.

```
In [42]: plt.figure(figsize=(6,4))

plt.scatter(df["Study_Hours"], df["Marks"],
            color="green",
            s=100)

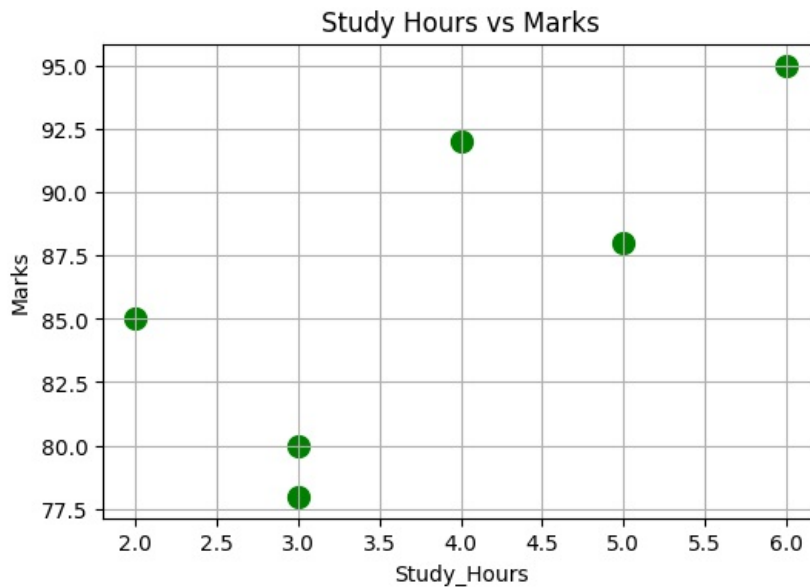
plt.title("Study Hours vs Marks")

plt.xlabel("Study_Hours")

plt.ylabel("Marks")

plt.grid(True)

plt.show()
```



Observation

The scatter plot shows the relationship between study hours and marks. Students who studied for more hours generally achieved higher marks. Although the dataset is small, the graph indicates a positive relationship between study time and academic performance.

5.3 Bar Chart

Description

A bar chart represents data using rectangular bars, where the height of each bar corresponds to its value. It is one of the most effective ways to compare values across different categories.

Use Case

Bar charts are commonly used to compare sales of products, students' marks, monthly expenses, or any data that belongs to different categories.

```
In [43]: plt.figure(figsize=(6,4))

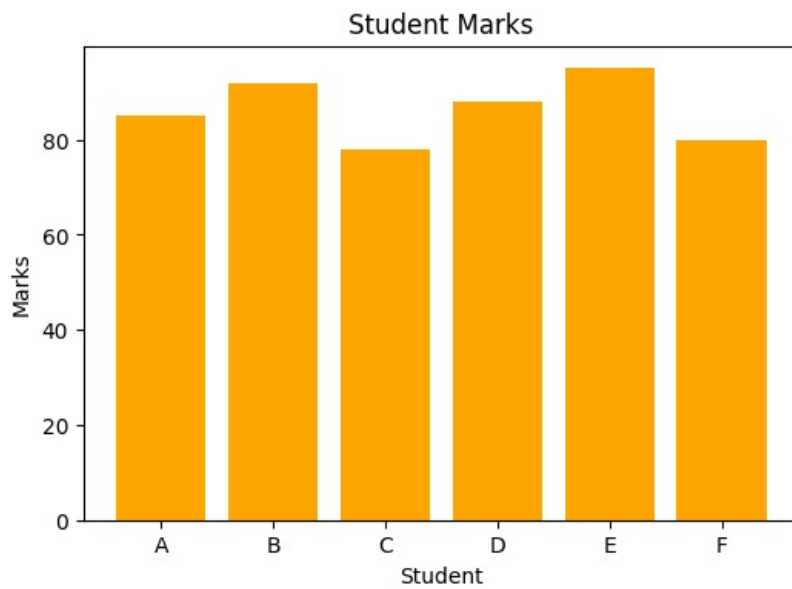
plt.bar(df["Student"], df["Marks"],
        color="orange")

plt.title("Student Marks")

plt.xlabel("Student")

plt.ylabel("Marks")

plt.show()
```



Observation

The bar chart compares the marks of all students. It clearly shows that Student E scored the highest marks, while Student C scored the lowest. The differences between students' performances are easy to identify through the height of the bars.

5.4 Histogram

Description

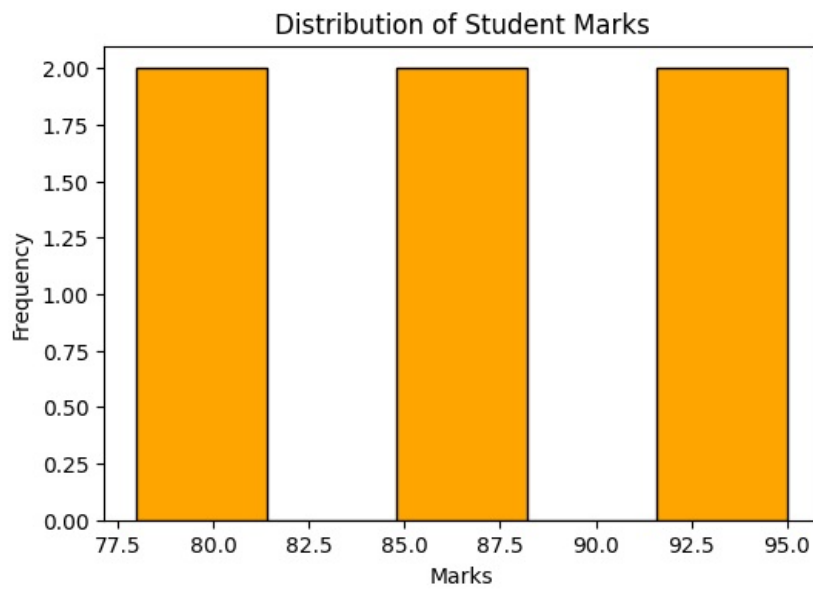
A histogram is used to display the distribution of numerical data by grouping values into intervals, called bins. It helps understand how the data is spread and whether there are any patterns or concentrations.

Use Case

Histograms are commonly used to analyze the distribution of marks, ages, salaries, heights, or any continuous numerical data.

```
In [44]: plt.figure(figsize=(6,4))
plt.hist(df["Marks"],
         bins=5,
         color="orange",
         edgecolor="black")

plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Frequency")
plt.show()
```



Observation

The histogram shows how students' marks are distributed across different ranges. Most students scored between **80 and 95 marks**, indicating that the overall performance of the class is good.

5.5 Pie Chart

Description

A pie chart represents data as slices of a circle, where each slice shows the proportion of a category relative to the whole. It is useful for displaying percentages or shares.

Use Case

Pie charts are commonly used to show budget allocation, market share, survey results, or the percentage contribution of different categories.

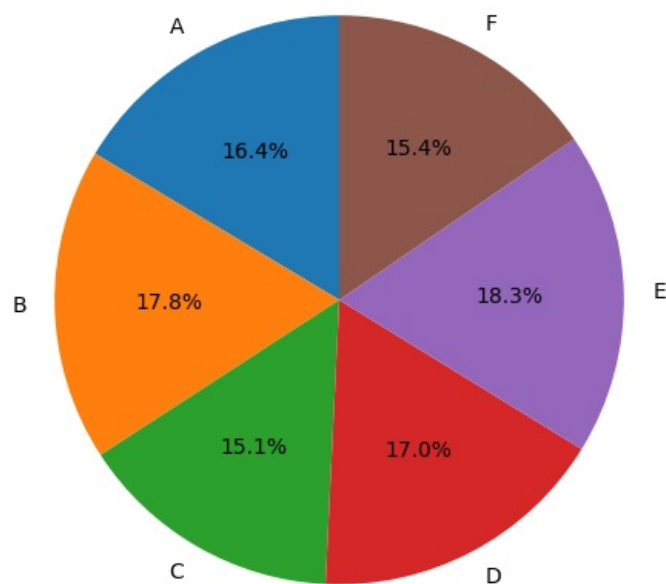
```
In [45]: plt.figure(figsize=(6,6))

plt.pie(df["Marks"],
        labels=df["Student"],
        autopct="%1.1f%%",
        startangle=90)

plt.title("Percentage of Marks by Student")
```

```
Out[45]: Text(0.5, 1.0, 'Percentage of Marks by Student')
```

Percentage of Marks by Student



Observation

The pie chart shows each student's contribution to the total marks. Student E occupies the largest portion of the chart because they scored the highest marks, while Student C has the smallest share.

5.6 Box Plot

Description

A box plot is used to summarize the distribution of numerical data. It displays the minimum value, first quartile (Q1), median, third quartile (Q3), and maximum value. It is also useful for identifying outliers.

Use Case

Box plots are commonly used to compare the spread of data, detect outliers, and understand the distribution of marks, salaries, or other numerical values.

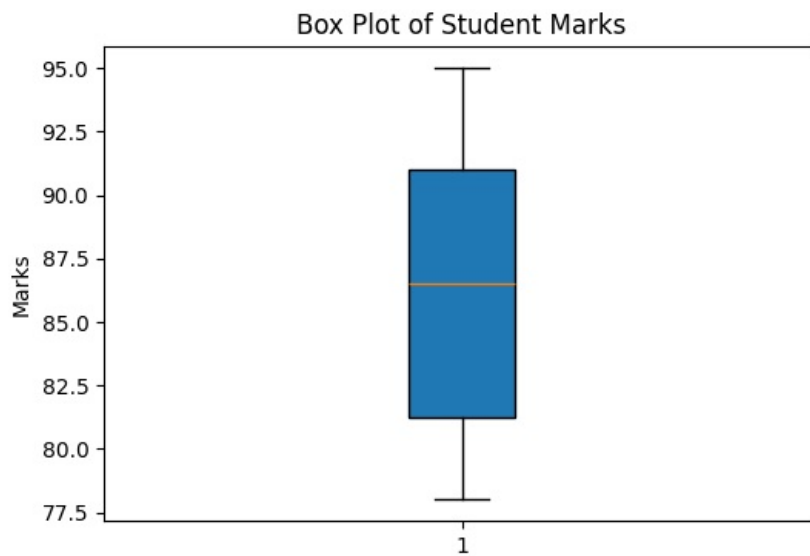
```
In [46]: import matplotlib.pyplot as plt  
plt.figure(figsize=(6,4))
```



```
plt.boxplot(df["Marks"], patch_artist=True)

plt.title("Box Plot of Student Marks")
plt.ylabel("Marks")

plt.show()
```



Observation

The box plot displays the spread of students' marks. The line inside the box represents the median, while the box shows the middle 50% of the data. Since there are no points outside the whiskers, there are no significant outliers in this dataset.

5.7 Area Chart

Description

An area chart is similar to a line chart, but the area below the line is filled with color. It helps highlight trends and the magnitude of values over a period.

Use Case

Area charts are commonly used to visualize changes over time, such as sales, website traffic, rainfall, or student performance.

```
In [47]: import matplotlib.pyplot as plt

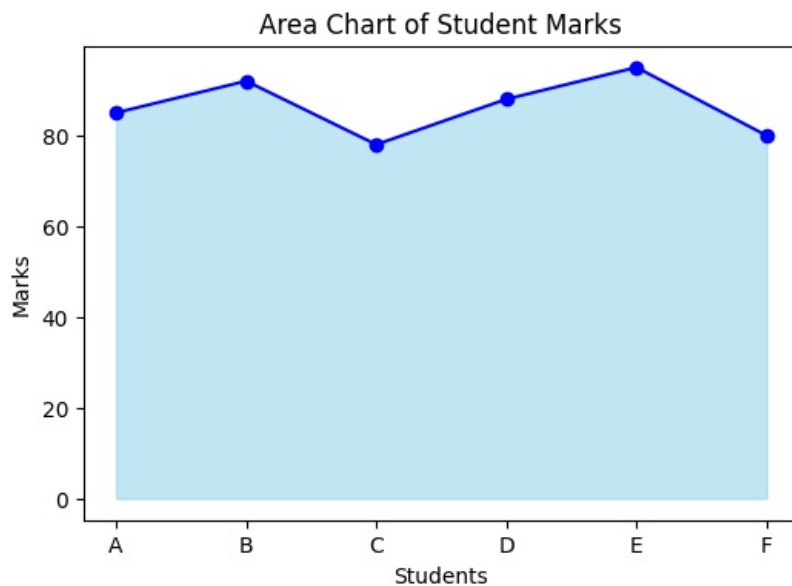
plt.figure(figsize=(6,4))

plt.fill_between(
    df["Student"],
    df["Marks"],
    color="skyblue",
    alpha=0.5
)

plt.plot(
    df["Student"],
    df["Marks"],
    marker="o",
    color="blue"
)

plt.title("Area Chart of Student Marks")
plt.xlabel("Students")
plt.ylabel("Marks")

plt.show()
```



Observation

The area chart shows the marks scored by each student while highlighting the area beneath the line. It makes it easy to observe the overall trend, with Student E achieving the highest marks.

6. Overview of Seaborn

Seaborn is a popular Python library used for creating attractive and informative data visualizations. It is built on top of Matplotlib, which means it uses Matplotlib's capabilities while providing a simpler and cleaner interface. With just a few lines of code, users can create professional-looking charts that are useful for exploring and understanding data.

Key Features of Seaborn

- Built on top of Matplotlib.
- Easy to learn and use.
- Attractive default themes and colors.
- Supports statistical visualizations.
- Works well with Pandas DataFrames.
- Ideal for exploratory data analysis (EDA).

Installing and Importing Seaborn

Google Colab already includes Seaborn, but the installation command is shown below for reference.

```
In [48]: !pip install seaborn
```

Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
 Requirement already satisfied: numpy!=1.24.0,>=1.20 in /usr/local/lib/python3.12/dist-packages (from seaborn) (2.0.2)
 Requirement already satisfied: pandas>=1.2 in /usr/local/lib/python3.12/dist-packages (from seaborn) (2.2.2)
 Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /usr/local/lib/python3.12/dist-packages (from seaborn) (3.10.0)
 Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
 Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.63.0)
 Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.0)
 Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.2)
 Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.3.0)
 Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)
 Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
 Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2->seaborn) (2025.2)
 Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.2->seaborn) (2026.2)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)

```
In [49]: import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
```

6.1 Scatter Plot

Description

A scatter plot displays individual data points to show the relationship between two numerical variables.

Use Case

It is useful for checking whether two variables are related, such as study hours and exam marks.

```
In [50]: df.head()
```

```
Out[50]:
```

	Student	Marks	Study_Hours	Attendance	Activity
0	A	85	2	90	Yes
1	B	92	4	95	Yes
2	C	78	3	85	No
3	D	88	5	92	Yes
4	E	95	6	98	No

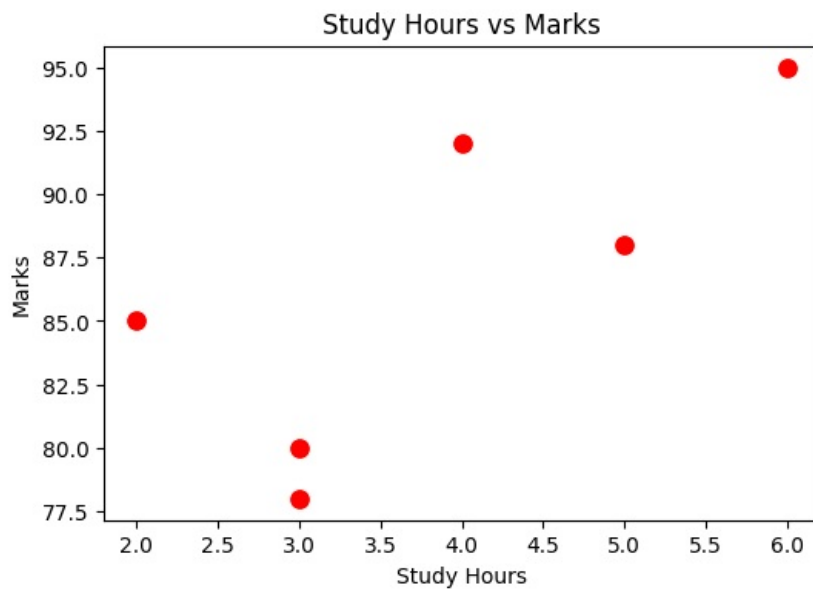
```
In [51]: data = {
    "Student": ["A", "B", "C", "D", "E", "F"],
    "Marks": [85, 92, 78, 88, 95, 80],
    "Study_Hours": [2, 4, 3, 5, 6, 3],
    "Attendance": [90, 95, 85, 92, 98, 88]
}
```

```
In [52]: plt.figure(figsize=(6,4))

sns.scatterplot(
    data=df,
    x="Study_Hours",
    y="Marks",
    color="red",
    s=100
)

plt.title("Study Hours vs Marks")
plt.xlabel("Study Hours")
plt.ylabel("Marks")

plt.show()
```



Observation

The graph indicates that students who studied for more hours generally scored higher marks.

6.2 Line Plot

Description

A line plot connects data points using straight lines, making it easy to identify trends and changes.

Use Case

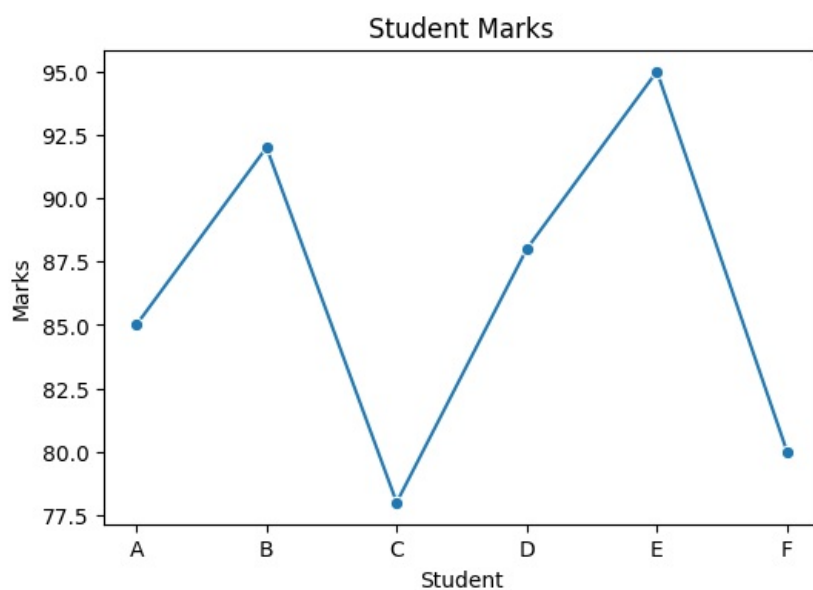
It is commonly used to monitor changes over time, such as monthly sales or student performance.

```
In [53]: plt.figure(figsize=(6,4))

sns.lineplot(
    data=df,
    x="Student",
    y="Marks",
    marker="o"
)

plt.title("Student Marks")

plt.show()
```



Observation

The line plot clearly shows the variation in marks among different students.

6.3 Bar Plot

Description

A bar plot compares numerical values across different categories using rectangular bars.

Use Case

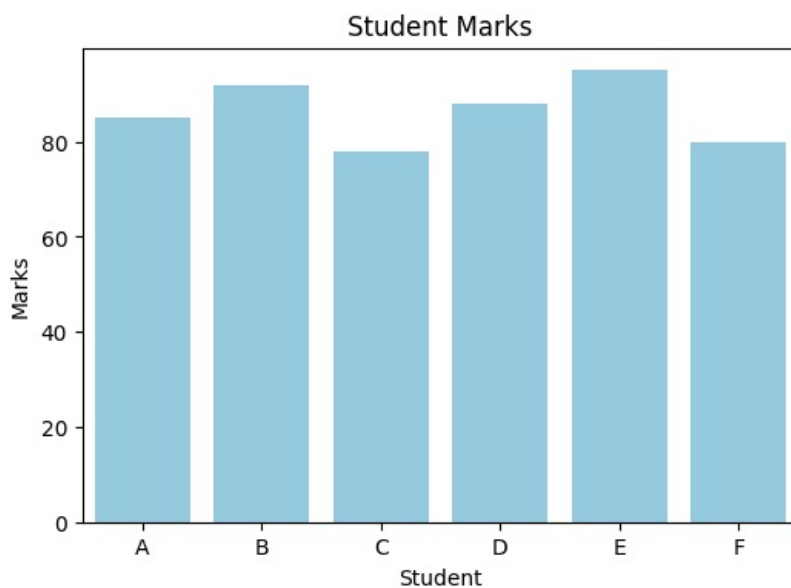
It is useful for comparing values such as marks, sales, or population across categories.

```
In [54]: plt.figure(figsize=(6,4))

sns.barplot(
    data=df,
    x="Student",
    y="Marks",
    color="skyblue"
)

plt.title("Student Marks")

plt.show()
```



Observation

The bar plot makes it easy to compare the marks scored by each student.

6.4 Histogram

Description

A histogram groups numerical data into intervals to show how the values are distributed.

Use Case

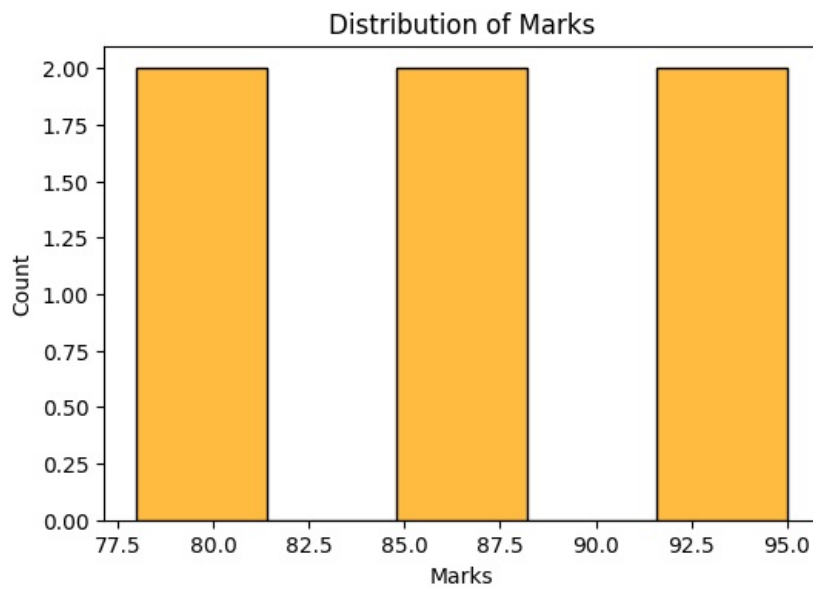
Histograms are useful for understanding the distribution of marks, ages, salaries, or other numerical data.

```
In [55]: plt.figure(figsize=(6,4))

sns.histplot(
    data=df,
    x="Marks",
    bins=5,
    color="orange"
)

plt.title("Distribution of Marks")

plt.show()
```



Observation

The histogram shows that most students scored between 80 and 95 marks.

6.5 Box Plot

Description

A box plot summarizes the spread of numerical data and helps identify outliers.

Use Case

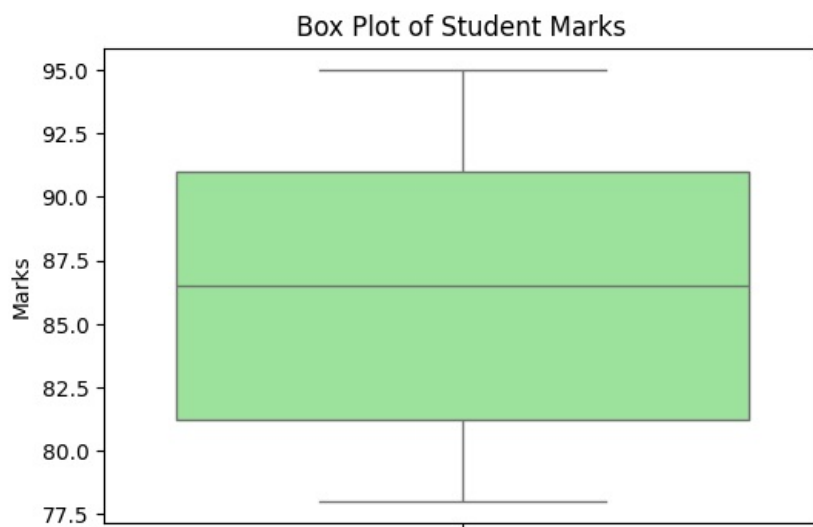
It is commonly used to compare distributions and detect unusual values in a dataset.

```
In [56]: plt.figure(figsize=(6,4))

sns.boxplot(
    data=df,
    y="Marks",
    color="lightgreen"
)

plt.title("Box Plot of Student Marks")

plt.show()
```



Observation

The box plot shows the median and spread of marks. There are no significant outliers in this dataset.

6.6 Heatmap

Description

A heatmap uses different colors to represent values in a matrix. It is commonly used to visualize correlations between numerical variables.

Use Case

Heatmaps help identify positive or negative relationships between different variables in a dataset.

```
In [58]: df.head()
```

```
Out[58]:
```

	Student	Marks	Study_Hours	Attendance	Activity
0	A	85	2	90	Yes
1	B	92	4	95	Yes
2	C	78	3	85	No
3	D	88	5	92	Yes
4	E	95	6	98	No

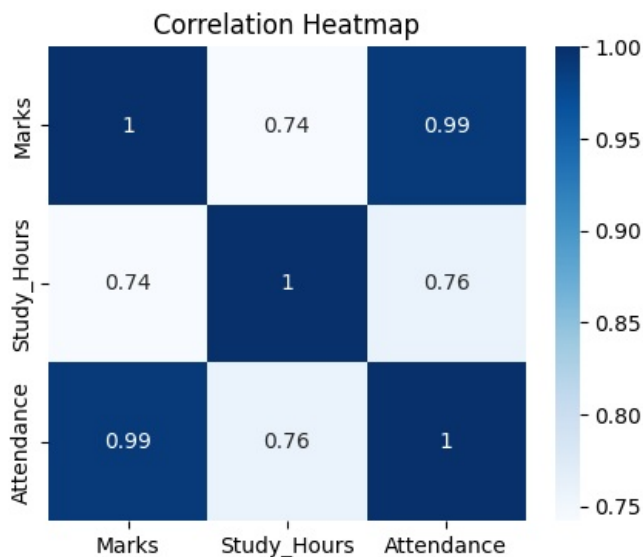
```
In [59]: plt.figure(figsize=(5,4))

correlation = df[["Marks", "Study_Hours", "Attendance"]].corr()

sns.heatmap(
    correlation,
    annot=True,
    cmap="Blues"
)

plt.title("Correlation Heatmap")

plt.show()
```



Observation

The heatmap shows that study hours and attendance are positively related to students' marks.

7. Comparison Between Matplotlib and Seaborn

Matplotlib

Matplotlib is a powerful visualization library that provides complete control over creating graphs. It supports many chart types and is best suited for users who need detailed customization.

Advantages:

- Highly customizable

- Supports many graph types
- Works well with NumPy and Pandas
- Ideal for detailed visualizations

Seaborn

Seaborn is built on top of Matplotlib and makes it easier to create attractive statistical graphs. It offers better default styles and requires less code.

Advantages:

- Easy to use
- Attractive default themes
- Excellent for statistical plots
- Integrates well with Pandas

Overall Comparison

Matplotlib is best for creating customized visualizations, while Seaborn is better for quickly creating clean and informative statistical graphs. Both libraries are widely used together in data science projects.

8. Conclusion

Matplotlib and Seaborn are both powerful libraries for data visualization in Python. Matplotlib provides greater flexibility and control for creating customized graphs, while Seaborn offers a simpler approach with attractive default styles and statistical plotting features. Choosing the right library depends on the type of visualization and the level of customization required.

9. References

References

1. Matplotlib Documentation: https://matplotlib.org/stable/users/explain/quick_start.html
2. Seaborn Documentation: <https://seaborn.pydata.org/tutorial/introduction.html>